Module 1 Topic Map from OCR Questions

OCR sources used only: May 2024 paper, June 2023 paper, and DC+SC OCR output (including full file scan; pages 7 and 8 verified while collecting context).

Frequently repeated concepts

- Artificial neuron model and net input computation.
- Biological neuron vs artificial neuron.
- Activation functions (binary sigmoid, bipolar sigmoid) and role.
- McCulloch-Pitts (MP) neuron for logic gates.
- Hebb learning rule, training steps, and logic implementation.
- Linear separability; XOR as non-linearly separable.

High-mark concepts

- Linear separability + AND with Hebb + XOR justification.
- ANDNOT using MP neuron with architecture and threshold conditions.
- Hebb weight derivation for class patterns and testing.
- Neuron output numericals using sigmoid functions with bias.

Numerical/problem patterns

- Net input: $v = \sum_i w_i x_i + b$.
- Sigmoid output: binary $f(v) = \frac{1}{1+e^{-v}}$, bipolar $g(v) = \frac{1-e^{-v}}{1+e^{-v}}$.
- Hebb update: $\Delta w_i = \eta x_i t$ (batch form: $w_i = \sum_p x_i^{(p)} t^{(p)}$).

Diagram-heavy asks

- Artificial neuron block.
- MP neuron for AND/ANDNOT.
- Decision boundary idea for linear separability.

Must-study-first order for first-time learners

1. Neuron basics: input, weight, bias, net input, activation.
2. Activation functions and why nonlinearity matters.
3. MP neuron for logic gates.
4. Hebb learning and simple weight calculations.
5. Linear separability and XOR limitation.
6. Short ANN architecture and learning method classifications.

Core Theory (Teach-First, Exam-Ready)

1) Artificial Neuron from Zero

Intuition: A neuron is a weighted decision unit. Important inputs get higher weights, and bias shifts decision strictness.

Formal definition:

$$v = \sum_{i=1}^{n} w_i x_i + b, \qquad y = \phi(v)$$

where x_i are inputs, w_i are synaptic weights, b is bias, v is net input, and $\phi(\cdot)$ is activation.

Simple Artificial Neuron

$$x_1, x_2, \ldots, x_n \xrightarrow{w_1, w_2, \ldots, w_n} \text{Weighted Sum} + b \to \text{Activation } \phi \to y$$

Mini example: If $x = [0.5, 0.3]$, $w = [0.3, 0.7]$, $b = 0$, then

$$v = 0.5(0.3) + 0.3(0.7) = 0.36.$$

2) Why Activation Function Matters

Intuition: Without activation, multiple layers still behave like one linear model.

Two common functions

- Binary sigmoid: $f(v) = \dfrac{1}{1 + e^{-v}}$ with range $(0, 1)$.
- Bipolar sigmoid: $g(v) = \dfrac{1 - e^{-v}}{1 + e^{-v}} = \dfrac{2}{1 + e^{-v}} - 1$ with range $(-1, 1)$.

Where used: Classification, smooth gradient-based learning, confidence-like outputs.

3) Biological Neuron vs Artificial Neuron

- Biological dendrites $\leftrightarrow$ ANN inputs.
- Biological synaptic strength $\leftrightarrow$ ANN weights.
- Axon output spike $\leftrightarrow$ ANN activation output.
- Biological processing is electrochemical and massively parallel; ANN is mathematical and programmable.

4) McCulloch-Pitts (MP) Neuron

Definition: A threshold logic neuron with binary input-output.

$$y = \begin{cases} 1, & \sum_i w_i x_i \geq \theta \\ 0, & \text{otherwise} \end{cases}$$

AND gate setup: $w_1 = w_2 = 1,\ \theta = 2.$

ANDNOT setup ($x_1 \wedge \neg x_2$): $w_1 = 1,\ w_2 = -1,\ \theta = 1.$

5) Hebb Learning Rule

Intuition: "Neurons that fire together wire together."

Rule:

$$\Delta w_i = \eta x_i t$$

For one-pass batch design in logic tasks (often used in exams):

$$w_i = \sum_p x_i^{(p)} t^{(p)}, \qquad w_0 = \sum_p t^{(p)} \quad (x_0 = 1)$$

Common mistakes

- Mixing binary and bipolar coding in same derivation.
- Forgetting bias term when asked for biased neuron.
- Using wrong target sign for NOR/NAND style tasks.

Exam Tip: For Hebb numericals, write coding first (binary or bipolar), then table, then weight sums. This fetches method marks even if arithmetic slips.

6) Linear Separability and XOR

Definition: If a single line $w_1x_1 + w_2x_2 + b = 0$ can split two classes, the problem is linearly separable.

AND is separable, XOR is not. XOR needs at least one hidden layer or nonlinear feature mapping.

Decision Boundary Idea

Single line works for AND / OR; no single line can separate XOR's diagonal class pattern.

7) Architectures and Learning Types (Frequently Asked Theory)

ANN architectures (connection-based, exam list):

- Single-layer feedforward
- Multi-layer feedforward
- Recurrent/feedback network
- Fully connected network
- Lateral connection network

Learning methods:

- Supervised learning
- Unsupervised learning
- Reinforcement learning

Soft vs Hard computing (short contrast):

- Hard computing: exact model, deterministic, crisp logic.
- Soft computing: tolerance for uncertainty, approximate reasoning, learning/adaptation (fuzzy, ANN, evolutionary methods).

Numericals and Derivations (Fully Worked)

N1) Sigmoid output for $x_1 = 0.7, x_2 = 0.8, w_1 = 0.2, w_2 = 0.3, w_b = 0.9$

Given

- Inputs: $x_1 = 0.7$, $x_2 = 0.8$, bias input $x_0 = 1$.
- Weights: $w_1 = 0.2$, $w_2 = 0.3$, bias weight $w_b = 0.9$.

Find

- Output using binary sigmoid and bipolar sigmoid.

Step 1: Net input formula

$$v = \sum_i w_i x_i + w_b x_0$$

Step 2: Substitute values

$$v = 0.2(0.7) + 0.3(0.8) + 0.9(1) = 0.14 + 0.24 + 0.90 = 1.28$$

Step 3: Binary sigmoid formula

$$y_{bin} = \frac{1}{1 + e^{-v}} = \frac{1}{1 + e^{-1.28}}$$

Using $e^{-1.28} \approx 0.2780$,

$$y_{bin} = \frac{1}{1 + 0.2780} = \frac{1}{1.2780} \approx 0.7825$$

Step 4: Bipolar sigmoid formula

$$y_{bip} = \frac{1 - e^{-v}}{1 + e^{-v}} = \frac{1 - 0.2780}{1 + 0.2780} = \frac{0.7220}{1.2780} \approx 0.5650$$

Final answer

- Binary sigmoid output: 0.7825.
- Bipolar sigmoid output: 0.5650.

Sanity check: Since v is positive and reasonably large, both outputs must be positive and above midpoint. Correct.

N2) 4-input linear neuron output

Given

- Inputs: $x = [4, 10, 5, 20]$.
- Weights: $w = [1, 2, 3, 4]$.
- Linear transfer function: $y = 2v$.

Find: output y.

Step 1: Net input formula

$$v = \sum_{i=1}^{4} w_i x_i$$

Step 2: Substitute each term

$$v = 1(4) + 2(10) + 3(5) + 4(20) = 4 + 20 + 15 + 80 = 119$$

Step 3: Apply transfer function

$$y = 2v = 2(119) = 238$$

Final answer: $y = 238$.

N3) Hebb design for NOR with bipolar coding

Given

- Bipolar input coding: -1 for logic 0, $+1$ for logic 1.
- Target for NOR: $t = +1$ only when $(x_1, x_2) = (-1, -1)$, else $t = -1$.

Find: Hebb weights (w_0, w_1, w_2).

Step 1: Write training table

x_1	x_2	t	x_0
-1	-1	$+1$	1
-1	$+1$	-1	1
$+1$	-1	-1	1
$+1$	$+1$	-1	1

Step 2: Hebb batch formulas

$$w_1 = \sum t x_1, \qquad w_2 = \sum t x_2, \qquad w_0 = \sum t x_0$$

Step 3: Compute each weight

$$w_1 = (+1)(-1) + (-1)(-1) + (-1)(+1) + (-1)(+1) = -2$$

$$w_2 = (+1)(-1) + (-1)(+1) + (-1)(-1) + (-1)(+1) = -2$$
$$w_0 = (+1)(1) + (-1)(1) + (-1)(1) + (-1)(1) = -2$$

Step 4: Final decision rule

$$y = \text{sgn}(w_0 + w_1x_1 + w_2x_2) = \text{sgn}(-2 - 2x_1 - 2x_2)$$

Final answer: $w_0 = w_1 = w_2 = -2$, which correctly realizes NOR in bipolar form.

Solved Questions for Module 1 (Self-Contained)

Part A / 3-Mark Questions (Each Answer Gives 6 Scoring Points)

Q1. Role of activation function in ANN. Explain any two activation functions.

1. Activation function converts net input into final neuron output.
2. It introduces nonlinearity; without it, multilayer ANN reduces to linear mapping.
3. It keeps output in bounded range, improving numerical stability.
4. Binary sigmoid: $f(v) = \dfrac{1}{1+e^{-v}}$, output range $(0, 1)$.
5. Bipolar sigmoid: $g(v) = \dfrac{1-e^{-v}}{1+e^{-v}}$, output range $(-1, 1)$.
6. Binary sigmoid suits probability-like output; bipolar sigmoid is useful for bipolar target coding.

Q2. Draw simple artificial neuron and discuss net input calculation.

1. Inputs $x_1, x_2, \ldots, x_n$ enter through weighted links $w_1, w_2, \ldots, w_n$.
2. Bias b shifts the decision threshold.
3. Net input is computed as weighted sum plus bias.
4. Formula: $v = \sum_{i=1}^{n} w_i x_i + b$.
5. Output is obtained by activation: $y = \phi(v)$.
6. Larger positive v usually gives stronger positive output for sigmoid/threshold units.

Q3. Compare biological neuron and artificial neuron.

1. Biological dendrites correspond to ANN input lines.
2. Biological synaptic strength corresponds to ANN weights.
3. Biological soma integration corresponds to weighted summation in ANN.
4. Biological axon firing corresponds to ANN output after activation.

5. Biological neurons adapt through biochemical plasticity; ANN adapts by algorithmic weight updates.
6. Biological network is noisy and complex; ANN model is simplified and mathematically tractable.

Q4. Differentiate soft computing and hard computing.

1. Hard computing aims exact solution with strict deterministic model.
2. Soft computing allows approximation when data is uncertain or noisy.
3. Hard computing uses crisp true/false logic; soft computing handles graded truth and uncertainty.
4. Hard methods need precise mathematical model; soft methods can learn from data.
5. Hard systems are brittle under ambiguity; soft systems are robust under imprecision.
6. ANN, fuzzy logic, and GA are classic soft-computing paradigms.

Q5. Explain five basic ANN architectures based on connection.

1. Single-layer feedforward: input directly to output layer.
2. Multilayer feedforward: one or more hidden layers for nonlinear mapping.
3. Recurrent/feedback network: includes loops, supports dynamic memory.
4. Fully connected network: each neuron in one layer connects to all neurons in next layer.
5. Lateral connection network: neurons in same layer can influence neighbors.
6. Architecture choice depends on task complexity, temporal behavior, and data structure.

Q6. What are the learning methods employed in neural networks?

1. Supervised learning uses labeled input-output pairs.
2. Unsupervised learning discovers hidden structure without target labels.
3. Reinforcement learning uses reward/penalty feedback from environment.
4. Supervised examples: Perceptron, Adaline, backpropagation.
5. Unsupervised examples: clustering and competitive learning.
6. Reinforcement is useful when exact target is unavailable but performance feedback exists.

Q7. "Hebb rule is more suited for bipolar data than binary data." Justify.

1. Hebb update uses product $x_i t$, so sign carries meaningful correlation.
2. Bipolar coding naturally represents excitation (+) and inhibition (−).

3. Binary coding cannot express negative correlation directly.
4. Bipolar representation produces symmetric decision boundaries around zero.
5. Mathematical updates become cleaner for logic functions like NOR and XNOR.
6. Hence bipolar coding usually gives more balanced Hebbian learning behavior.

Q8. Explain Hebb training algorithm.

1. Initialize all weights (and bias) to zero or small values.
2. Present one input-target pair (x, t).
3. Compute weight change by $\Delta w_i = \eta x_i t$.
4. Update each weight: $w_i \leftarrow w_i + \Delta w_i$.
5. Repeat for all features and all patterns in training set.
6. Stop after required epochs or when stable classification behavior is achieved.

Q9. Implement AND using McCulloch-Pitts neuron (binary).

1. Use binary inputs $x_1, x_2 \in \{0, 1\}$.
2. Choose weights $w_1 = 1$ and $w_2 = 1$.
3. Choose threshold $\theta = 2$.
4. Decision rule: output 1 if $w_1 x_1 + w_2 x_2 \geq \theta$ else 0.
5. Only $(1, 1)$ gives sum 2, so output is 1 only for this case.
6. Therefore neuron exactly implements AND gate truth table.

Q10. Implement ANDNOT using McCulloch-Pitts neuron.

1. Required logic: $y = x_1 \wedge \neg x_2$.
2. Choose weights $w_1 = 1$ and $w_2 = -1$.
3. Choose threshold $\theta = 1$.
4. Net expression: $v = w_1 x_1 + w_2 x_2 = x_1 - x_2$.
5. Only input $(x_1, x_2) = (1, 0)$ satisfies $v \geq 1$.
6. Hence output becomes 1 only for ANDNOT condition.

Q11. Calculate net input for given two-input neuron (figure-type numerical).

1. Write net input formula: $v = \sum w_i x_i + b$.
2. If no bias is shown, use $b = 0$.

3. For OCR model values: $x_1 = 0.5, x_2 = 0.3, w_1 = 0.3, w_2 = 0.7$.
4. Substitute: $v = 0.3(0.5) + 0.7(0.3)$.
5. Compute terms: $0.15 + 0.21 = 0.36$.
6. Final answer: net input is $v = 0.36$.

Q12. Find output for $x = [0.7, 0.8]$, $w = [0.2, 0.3]$, bias weight 0.9 using binary and bipolar sigmoid.

1. Compute net input using $v = w_1x_1 + w_2x_2 + w_b$.
2. Substitution: $v = 0.2(0.7) + 0.3(0.8) + 0.9 = 1.28$.
3. Binary sigmoid formula: $y_{bin} = \dfrac{1}{1 + e^{-1.28}}$.
4. Numerically, $y_{bin} \approx 0.7825$.
5. Bipolar sigmoid formula: $y_{bip} = \dfrac{1 - e^{-1.28}}{1 + e^{-1.28}}$.
6. Numerically, $y_{bip} \approx 0.5650$.

Q13. Design Hebb net for NOR using bipolar inputs and targets.

1. Use bipolar coding: logic 0 as -1, logic 1 as $+1$.
2. NOR target is $+1$ only for $(-1, -1)$ and -1 otherwise.
3. Use Hebb batch equations: $w_1 = \sum tx_1$, $w_2 = \sum tx_2$, $w_0 = \sum tx_0$.
4. Compute: $w_1 = -2$, $w_2 = -2$, $w_0 = -2$.
5. Decision function: $y = \text{sgn}(w_0 + w_1x_1 + w_2x_2)$.
6. This function reproduces NOR truth behavior in bipolar form.

Q14. 4-input linear transfer numerical: predict output.

1. Given $w = [1, 2, 3, 4]$ and $x = [4, 10, 5, 20]$.
2. Net formula: $v = \sum w_ix_i$.
3. Substitute: $v = 1 \cdot 4 + 2 \cdot 10 + 3 \cdot 5 + 4 \cdot 20$.
4. Simplify: $v = 4 + 20 + 15 + 80 = 119$.
5. Transfer function is $y = 2v$.
6. Final answer: $y = 2(119) = 238$.

Part B / 14-Mark Questions (Each Answer Gives 14 Scoring Points)

Q1. Define linear separability. Justify XOR is non-linearly separable. Also explain AND implementation idea.

1. Linear separability means one straight line can separate two classes in 2D.
2. General decision boundary is $w_1x_1 + w_2x_2 + b = 0$.
3. Points with positive net value lie on one side; negative on the other side.
4. AND truth table has one positive class point $(1, 1)$ and three negative points.
5. A line such as $x_1 + x_2 = 1.5$ separates AND classes correctly.
6. So AND is linearly separable.
7. XOR has positive points at $(1, 0)$ and $(0, 1)$.
8. XOR has negative points at $(0, 0)$ and $(1, 1)$.
9. Positive and negative classes are diagonally interleaved.
10. Any single line that captures one positive point misclassifies the other region.
11. Therefore no single linear decision boundary can solve XOR.
12. XOR needs hidden layer or nonlinear feature mapping.
13. Hebb-trained single-layer neuron can solve AND but not XOR.
14. Conclusion: linear separability explains why simple perceptron succeeds for AND and fails for XOR.

Q2. Implement ANDNOT using MP neuron. Draw architecture and threshold conditions.

1. Required function is $y = x_1 \wedge \neg x_2$.
2. Use MP model with binary input and threshold output.
3. Choose excitatory weight for x_1: $w_1 = +1$.
4. Choose inhibitory weight for x_2: $w_2 = -1$.
5. Set threshold $\theta = 1$.
6. Net expression is $v = x_1 - x_2$.
7. Decision rule: output 1 if $v \geq 1$, else output 0.
8. For $(0, 0)$, $v = 0$, output 0.
9. For $(0, 1)$, $v = -1$, output 0.
10. For $(1, 0)$, $v = 1$, output 1.

11. For $(1,1)$, $v = 0$, output 0.
12. Truth table exactly matches ANDNOT.
13. Architecture uses two weighted links into one summing node and threshold unit.
14. Conclusion: opposite-sign weights naturally model excitation and inhibition.

ANDNOT MP Architecture

$$x_1 \xrightarrow{+1}$$
$$\Sigma \to \text{Threshold } \theta = 1 \to y$$
$$x_2 \xrightarrow{-1}$$

Q3. Using Hebb rule, derive weights for classifying I and O patterns and test the samples.

1. Convert each 3x3 character pixel into bipolar values: foreground $+1$, background -1.
2. Assign target $t = +1$ for class I and $t = -1$ for class O.
3. Write input vectors in flattened form $x^{(p)} = [x_1, \ldots, x_9]$.
4. Use Hebb batch formula for each feature: $w_j = \sum_p x_j^{(p)} t^{(p)}$.
5. Equivalent matrix form for two prototypes is $W = I - O$ (after bipolar coding).
6. One consistent OCR-style coding gives

$$I = \begin{bmatrix} 1 & 1 & 1 \\ -1 & 1 & -1 \\ 1 & 1 & 1 \end{bmatrix}, \quad O = \begin{bmatrix} 1 & 1 & 1 \\ 1 & -1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

7. Compute prototype difference:

$$W = I - O = \begin{bmatrix} 0 & 0 & 0 \\ -2 & 2 & -2 \\ 0 & 0 & 0 \end{bmatrix}$$

8. During testing, compute net value $v = \sum_{j=1}^{9} w_j x_j$.
9. If $v > 0$, classify as I.
10. If $v < 0$, classify as O.
11. Testing with I produces positive correlation because center column matches positive weights.
12. Testing with O produces negative response due to center-row mismatch pattern.
13. Thus learned weights separate the two character classes.
14. Conclusion: Hebbian correlation forms a simple associative classifier for prototype patterns.

Q4. Compute neuron output for given network using binary and bipolar sigmoid activation.

1. Given: $x_1 = 0.7$, $x_2 = 0.8$, $w_1 = 0.2$, $w_2 = 0.3$, bias weight $w_b = 0.9$, bias input $x_0 = 1$.
2. Required: compute outputs using binary sigmoid and bipolar sigmoid.
3. Net input formula: $v = w_1x_1 + w_2x_2 + w_bx_0$.
4. Substitute and simplify: $v = 0.2(0.7) + 0.3(0.8) + 0.9(1) = 1.28$.
5. Binary sigmoid formula: $y_{bin} = \dfrac{1}{1+e^{-v}}$.
6. Substitute: $y_{bin} = \dfrac{1}{1+e^{-1.28}}$.
7. Numerical approximation: $e^{-1.28} \approx 0.2780$.
8. Therefore $y_{bin} = \dfrac{1}{1+0.2780} = 0.7825$.
9. Bipolar sigmoid formula: $y_{bip} = \dfrac{1-e^{-v}}{1+e^{-v}}$.
10. Substitute: $y_{bip} = \dfrac{1-e^{-1.28}}{1+e^{-1.28}}$.
11. Using same exponential value: $y_{bip} = \dfrac{1-0.2780}{1+0.2780} = 0.5650$.
12. Range check: binary output must lie in $(0, 1)$ and bipolar in $(-1, 1)$.
13. Both results satisfy their ranges and are positive because net input is positive.
14. Final answer: $y_{bin} = 0.7825$, $y_{bip} = 0.5650$.

Formula Sheet (Module 1)

- Net input: $v = \sum_i w_i x_i + b$
- MP threshold neuron: $y = 1$ if $v \geq \theta$, else 0
- Binary sigmoid: $f(v) = \dfrac{1}{1+e^{-v}}$
- Bipolar sigmoid: $g(v) = \dfrac{1-e^{-v}}{1+e^{-v}} = \dfrac{2}{1+e^{-v}} - 1$
- Hebb update: $\Delta w_i = \eta x_i t$
- Batch Hebb weight: $w_i = \sum_p x_i^{(p)} t^{(p)}$
- Linear decision boundary: $w_1x_1 + w_2x_2 + b = 0$

One-Page Quick Revision Summary

Memory hooks

- **Neuron** = weighted vote + bias + activation.
- **Activation** = adds nonlinearity; otherwise deep network behaves linear.
- **MP neuron** = threshold logic (great for AND, OR, ANDNOT).
- **Hebb** = correlation learning (fire together, wire together).
- **Linearly separable** = one line can split classes; XOR fails this.

Exam writing skeleton (works for most 14-mark Module 1 answers)

1. Start with one-line definition.
2. Write governing equation(s).
3. Draw neat block/flow sketch.
4. Show truth table or worked numerical.
5. End with one-line interpretation.

Last-minute 60-second recall

- AND by MP: $w_1 = w_2 = 1$, $\theta = 2$.
- ANDNOT by MP: $(w_1, w_2, \theta) = (1, -1, 1)$.
- Sigmoid outputs stay bounded.
- XOR needs hidden layer; single line is impossible.
- In numericals, never skip: Given $\rightarrow$ Formula $\rightarrow$ Substitution $\rightarrow$ Final answer.

Exam Tip: If you forget details in exam, write the core equation first. Equations anchor marks even when wording is short.